UNIVERSIDADE KIMPA VITA

INSTITUTO POLITÉCNICO

DEPARTAMETO DE ENGENHARIA INFORMÁTICA

TRABALHO PRÁTICO DE SGBD II

TEMA: MODELAGEM E IMPLEMENTAÇÃO DE UM SISTEMA DE GESTÃO DE EVENTOS ACADÉMICOS

Curso: Engenharia Informática

Classe: 3º Ano

Período: Manhã

Grupo: 12 (CosmoDev)

ORIENTADOR

_____

MOYO KANIVENGIDIO

UNIVERSIDADE KIMPA VITA

INSTITUTO POLITÉCNICO

DEPARTAMETO DE ENGENHARIA INFORMÁTICA

TRABALHO PRÁTICO DE SGBD II

RELATÓRIO DE PESQUISA

Modelagem e Implementação de um Sistema de Gestão de Eventos Académicos

Membros integrantes:

1. Ancelmo Gomes Monteiro
2. Luntadila Gabriel
3. Pedro João Afonso
4. Matupa André Fernando Domingos
5. Pedro Viegas Armando

GitHub: https://github.com/CosmoDev-tech?tab=repositories

ÍNDICE

RESUMO EXECUTIVO 4

1. INTRODUÇÃO 1

1.2 Motivação do Projeto 1

2. OBJETIVOS 1

2.1 Objetivo Geral 1

1. Análise de Requisitos: 2

2. Design Conceptual – Modelo Entidade- Relacionamento (MER) 5

3. Design Lógico – Normalização e Esquema Relacional 6

4. Implementação da Base de Dados 8

5. Consultas e Relatórios 8

Reflexão Crítica 11

Desafio 1: Nomenclatura e Naming de Constraints 11

Desafio 2: IDENTITY e Inserção Controlada de Dados 11

Desafio 3: Agregação de Valores com STRING_AGG 12

Desafio 4: Tipos de Dados VARCHAR vs NVARCHAR 12

Desafio 5: Funções de Data e Formatação 12

CONCLUSÕES 13

RESUMO EXECUTIVO

Este relatório apresenta o desenvolvimento completo de um sistema de gestão de eventos académicos, desde a análise de requisitos até à implementação e validação de uma base de dados relacional. O projeto demonstra a aplicação prática de conceitos fundamentais de Sistemas de Gestão de Bases de Dados (SGBD), incluindo modelagem conceptual, normalização até à Terceira Forma Normal (3FN), implementação de restrições de integridade e desenvolvimento de consultas SQL otimizadas. O sistema foi implementado em SQLSERVER e validado com dados realistas, compreendendo cinco eventos, vinte participantes, dez oradores, nove sessões e trinta inscrições. Os resultados demonstram a eficácia da abordagem adoptada na resolução de problemas complexos de modelagem de dados e na garantia de consistência e integridade de informação.

Palavras-chave: SGBD, Modelagem de Dados, Normalização, SQL, Base de Dados Relacional, Integridade de Dados.

1. INTRODUÇÃO

A gestão de eventos académicos, como conferências, workshops e seminários, representa um desafio complexo que envolve a coordenação de múltiplas entidades interdependentes: eventos, participantes, oradores, sessões e inscrições. A falta de um sistema de gestão adequado pode resultar em inconsistências de dados, perda de informação crítica, dificuldades na geração de relatórios e ineficiência operacional. Um sistema de gestão de base de dados bem projetado é essencial para resolver estes problemas, garantindo que os dados sejam armazenados de forma eficiente, segura e consistente.

A importância de uma base de dados robusta é particularmente evidente em contextos académicos, onde a precisão, a rastreabilidade e a conformidade com regulamentações são críticas. Um sistema bem modelado permite não apenas armazenar informação, mas também extrair insights significativos através de consultas sofisticadas, facilitando a tomada de decisões informadas.

1.2 Motivação do Projeto

Este projeto foi concebido como um exercício prático de engenharia de software, com o objetivo de demonstrar a aplicação de princípios sólidos de design de bases de dados. A motivação central é ilustrar como uma análise cuidadosa de requisitos, seguida de uma modelagem rigorosa e uma implementação disciplinada, pode resultar num sistema que é simultaneamente funcional, eficiente e fácil de manter.

2. OBJETIVOS

2.1 Objetivo Geral

O objetivo geral deste projeto é projetar, implementar e validar uma base de dados relacional robusta para um sistema de gestão de eventos académicos, aplicando boas práticas de modelagem de dados, normalização e integridade de informação.

1. Análise de Requisitos:

a) O sistema de gestão de eventos académicos envolve as seguintes entidades:

Eventos — conferências, workshops, seminários

Participantes — pessoas que se inscrevem nos eventos

Oradores — especialistas que apresentam sessões

Sessões — blocos temáticos dentro de um evento

Inscrições — registo de um participante num evento

b) Aqui listamos todas as nossas entidades e os seus atributos importantes utilizando o Dicionário:

EVENTOS

Atributo	Tipo	Descrição
EventoID	INT (PK)	Identificador único do evento
Nome	VARCHAR(100)	Nome/título do evento
DataInicio	DATE	Data de início
DataFim	DATE	Data de término
Local	VARCHAR(100)	Local nome do recinto do evento
Capacidade	INT	Número máximo de participantes
PrecoInscricao	DECIMAL(10,2)	Preço de inscrição em Kz
Cidade	VARCHAR(100)	Cidade onde vai passar o Evento
TipoEvento	VARCHAR(50)	Tipo de Eventos.
Descricao	TEXT	Descrição detalhada do evento

PARTICIPANTES

Atributo	Tipo	Descrição
ParticipanteID	INT (PK)	Identificador único
Nome	VARCHAR(100)	Nome completo
Email	VARCHAR(60) UNIQUE	Email (único)
Telefone	VARCHAR(20)	Número de telefone
Instituicao	VARCHAR(200)	Instituição de origem
DataRegisto	DATETIME	Data em que ele faz o seu registro
DataNascimento	DATE	Data de nascimento

Orador

Atributo	Tipo	Descrição
OradorID	INT (PK)	Identificador único
Nome	VARCHAR(80)	Nome completo do orador
Email	VARCHAR(65) UNIQUE	Email (único)
Especialidade	VARCHAR(95)	Área de especialidade
Cidade	VARCHAR(50)	Cidade onde Orador Reside
Instituicao	VARCHAR(50)	Instituição/empresa

SESSÕES

Atributo	Tipo	Descrição
SessaoID	INT (PK)	Identificador único
EventoID	INT (FK)	Referência ao evento
OradorID	INT (FK)	Referência ao orador
Titulo	VARCHAR(60)	Título da sessão
DataHoraInicio	DATETIME	Data e hora de início
DataHoraFim	DATETIME	Data e hora de término
TipoSessao	VARCHAR(30)	Tipo de Sessões
Sala	VARCHAR(50)	Sala ou auditório

INSCRIÇÕES

Atributo	Tipo	Descrição
InscricaoID	INT (PK)	Identificador único
ParticipanteID	INT (FK)	Referência ao participante
EventoID	INT (FK)	Referência ao evento
DataInscricao	DATETIME	Data/hora da inscrição
StatusPagamento	VARCHAR(20)	Pendente/Pago/Cancelado
ValorPago	DECIMAL(10,2)	Valor efectivamente pago

c) Relações e Cardinalidades

Relação	Cardinalidade	Descrição
Evento → Sessoes	1:N	Um evento tem muitas sessões; cada sessão pertence a um evento.
Orador → Sessoes	1:N	Um orador pode apresentar em várias sessões.
Participante ↔ Evento	N:M	Resolvida pela entidade de associação Inscricoes.
Evento → Inscricoes	1:N	Um evento tem muitas inscrições.
Participante → Inscricoes	1:N	Um participante tem muitas inscrições.

d) Restrições de Integridade

Restrição	Implementação T-SQL
DataFim >= DataInicio em Eventos	CHECK (DataHoraFim > DataHoraInicio)
Capacidade > 0	CHECK (Capacidade > 0)
PrecoInscricao >= 0	CHECK (PrecoInscricao >= 0)
Email único — Participantes	UNIQUE (Email)
Email único — Oradores	UNIQUE (Email)
Sem dupla inscrição	UNIQUE (ParticipanteID, EventoID)
StatusPagamento	CHECK (Estado IN ('Pendente', ‘Pago’, 'Cancelada'))
ValorPago >= 0	CHECK (ValorPago >= 0)
DataHoraFim > DataHoraInicio em Sessoes	CHECK (DataHoraFim > DataHoraInicio)
Integridade referencial	FOREIGN KEY ... ON DELETE CASCADE / NO ACTION

2. Design Conceptual – Modelo Entidade- Relacionamento (MER)

O Modelo Entidade-Relacionamento foi desenvolvido utilizando notação de Chen e é apresentado na Figura 1. O diagrama ilustra claramente as cinco entidades, os seus atributos (com chaves primárias sublinhadas), os relacionamentos entre elas, e as cardinalidades de cada relacionamento.

O MER revela a estrutura fundamental do sistema: Evento é uma entidade central que se relaciona com Sessão (1:N) e Inscrição (1:N); Sessão relaciona-se também com Orador (N:1); e Inscrição resolve o relacionamento N:M entre Participante e Evento.

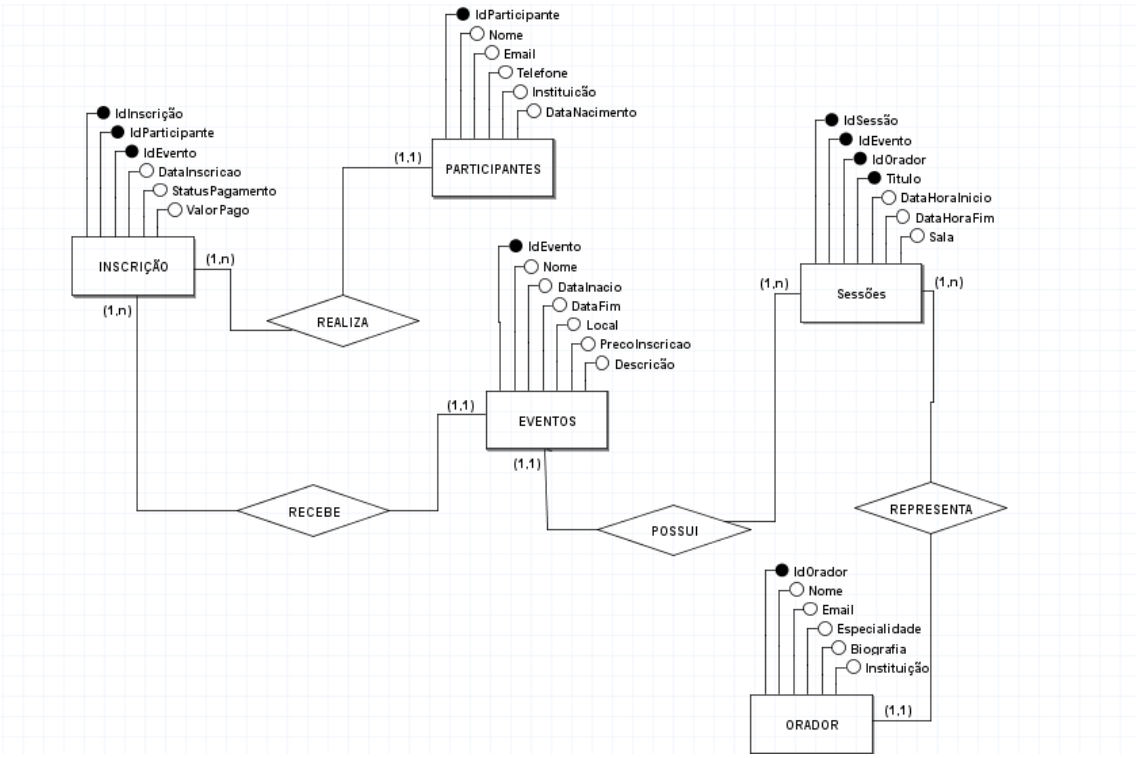

Figura 1 - Modelo Entidade-Relacionamento (BR Model)

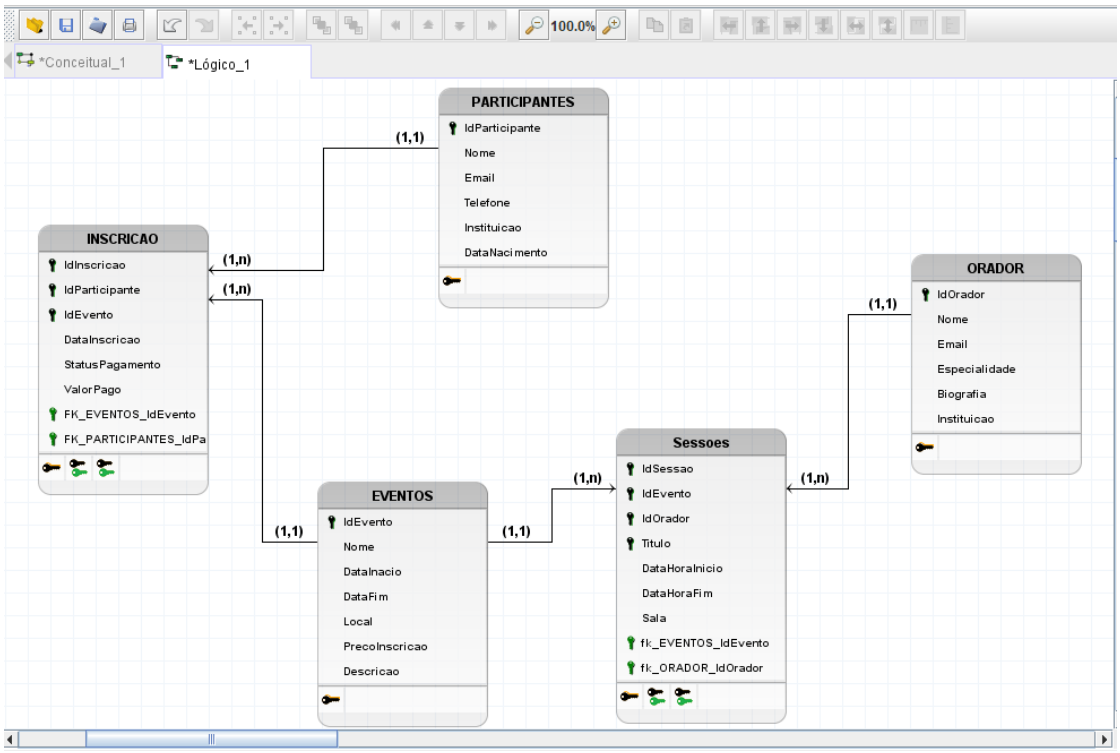

Figura 2 - Modelo Conceptual da BD

3. Design Lógico – Normalização e Esquema Relacional

Aplicação das Formas Normais

O processo de normalização foi aplicado sistematicamente para garantir que a base de dados estivesse na Terceira Forma Normal (3FN).

A normalização é o processo sistemático de decomposição de tabelas para eliminar a redundância de dados e as anomalias de inserção, atualização e eliminação. Aplicamos as três primeiras formas normais.

3.1. Processo de Normalização (1FN → 2FN → 3FN)

3.1.1. Primeira Forma Normal (1FN)

Todos os atributos devem ser atómicos (não divisíveis) e não pode haver grupos repetitivos de atributos. Cada célula da tabela deve conter um único valor.

- Todos os atributos identificados são atómicos (Nome, Email, DataInicio, etc.).
- Não existem atributos multivalorados ou grupos repetitivos nas nossas tabelas.
- PKs simples com IDENTITY(1,1).

Todas as tabelas satisfazem a 1FN desde a concepção.

3.1.2. Segunda Forma Normal (2FN)

Uma tabela está em 2FN se estiver em 1FN e todos os atributos não-chave forem totalmente dependentes da chave primária (sem dependências parciais).

- Como todas as chaves primárias são simples (surrogate keys), não há possibilidade de dependências parciais (que só ocorrem em chaves compostas).
    - Exemplo verificado: Em Inscricoes, todos os atributos (DataInscricao, StatusPagamento, ValorPago) dependem totalmente de InscricaoID. → Todas as tabelas satisfazem a 2FN.

3.1.3. Terceira Forma Normal (3FN)

Uma tabela está em 3FN se estiver em 2FN e não houver dependências transitivas de atributos não-chave em relação à chave primária.

Analisamos possíveis dependências transitivas em cada tabela:

Tabela	Dependência Analisada	Conclusão
Eventos	Local → Cidade?	Não. 'Local' = nome do recinto; 'Cidade' = localização geográfica. Ambos dependem de EventoID diretamente. 3FN
Sessoes	Sala → Titulo?	Não. A sala não determina o Titulo da sessão. Ambos dependem de SessaoID. 3FN
Participantes	Instituicao → ?	Instituição não determina outros atributos. Sem Cidade da Instituição para evitar transitividade. 3FN
Oradores	Instituicao → Especialidade?	Não. Especialidade é do orador, não derivada da instituição. 3FN
Inscricoes	StatusPag → ValorPago?	Não. ValorPago é valor efetivamente pago, independente do StatusPag. 3FN

3.2 Esquema Relacional Final Em 3FN

Tabela	Colunas e Tipos T-SQL
Eventos	(EventoID PK, Nome, DataInicio, DataFim, Local, Capacidade, PrecoInscricao, Cidade, Descricao)
Participantes	(ParticipanteID PK, Nome, Email UNIQUE, Telefone, Instituicao, DataNascimento, DataRegistro)
Oradores	(OradorID PK, Nome, Email UNIQUE, Especialidade, Biografia, Instituicao)
Sessoes	(SessaoID PK, EventoID FK→Eventos, OradorID FK→Oradores, Titulo, DataHoraInicio, DataHoraFim, Sala)
Inscricoes	(InscricaoID PK, ParticipanteID FK→Participantes, EventoID FK→Eventos, DataInscricao, StatusPagamento, ValorPago)

4. Implementação da Base de Dados

Os scripts DDL foram desenvolvidos em SQL SERVER 2025 EXPRESS e incluem a criação da base de dados, das tabelas, das chaves primárias e estrangeiras, e das restrições de integridade.

a) SQL Server é um **sistema de gestão de bases de dados (SGBD)** desenvolvido pela Microsoft, usado para **armazenar, organizar, consultar e proteger dados** de aplicações.

b) Comandos para criar a nossa Base de Dados, incluindo as chaves primarias e estrageiras, integridade e os outros requisitos:

c) O Script dos comandos DDL encontram se no repositório do GitHub do grupo e também no email do Engenheiro.

d) O Script dos comandos DML encontram se no repositório do GitHub do grupo e também no email do Engenheiro.

5. Consultas e Relatórios

a. Crie as seguintes consultas SQL (DML SELECT):

i. Listar todos os eventos com os seus oradores e o número total de participantes inscritos.

A consulta usa subconsultas correlacionadas para **agregar os oradores e contar participantes** de cada evento.

- `STRING_AGG` concatena os nomes dos oradores numa só célula, separados por "|", garantindo que **cada orador apareça apenas uma vez** por evento.
- `COUNT(DISTINCT)` conta os participantes únicos que têm `StatusPagamento = 'Pago'`, evitando duplicação caso um participante esteja inscrito em várias sessões do mesmo evento.
- O resultado é ordenado por `TotalParticipantes` em ordem decrescente, mostrando primeiro os eventos com mais inscritos pagos.

ii. Encontrar os participantes que se inscreveram em mais de um evento.

A consulta usa **INNER JOINs** para ligar participantes às suas inscrições e aos eventos correspondentes, incluindo apenas inscrições que **não estão canceladas** (`StatusPagamento <> 'Cancelado'`).

- `COUNT(i.EventoID)` conta quantos eventos cada participante está inscrito.
- `STRING_AGG` junta os nomes desses eventos numa só célula, separados por "|" e ordenados por data de início (`e.DataInicio`).
- `GROUP BY` agrupa os dados por participante, garantindo que cada um apareça apenas uma vez.
- O `HAVING COUNT(i.EventoID) > 1` filtra apenas os participantes que se inscreveram em **mais de um evento**.
- Por fim, `ORDER BY TotalEventos DESC` lista primeiro os participantes com mais inscrições.

iii. Obter a lista de sessões de um evento específico, ordenadas por data e hora.

A consulta usa **INNER JOINs** para ligar cada sessão ao seu orador e ao evento correspondente.

- `CONVERT` formata as datas de início e fim da sessão em `yyyy-mm-dd hh:mi`, facilitando a leitura.
- `DATEDIFF(MINUTE, s.DataHoraInicio, s.DataHoraFim)` calcula a **duração da sessão em minutos**.
- O `WHERE e.EventoID = 1` filtra as sessões de um evento específico (pode ser substituído por `LIKE` para filtrar pelo nome do evento).
- `ORDER BY s.DataHoraInicio ASC` organiza as sessões **cronologicamente**, do início ao fim do evento.

iv. Calcular a receita total gerada por cada evento.

A consulta usa um **LEFT JOIN** para incluir todos os eventos, mesmo aqueles sem inscrições confirmadas.

- `COUNT(CASE WHEN i.StatusPagamento = 'Pago' THEN 1 END)` conta apenas as inscrições **pagas**, ignorando pendentes ou canceladas.
- `ISNULL(SUM(...), 0)` calcula a **receita total** somando os valores pagos, retornando 0 se não houver pagamentos.
- `ROUND(..., 1)` calcula a **taxa de ocupação do evento** como percentual da capacidade, arredondando para 1 casa decimal.
- `GROUP BY` garante que cada evento apareça **uma única vez** com seus totais agregados.
- `ORDER BY ReceitaTotal DESC` lista primeiro os eventos que geraram **mais receita**.

v. Identificar oradores que participam em eventos em mais de uma cidade.

A consulta usa **INNER JOINs** para ligar oradores às suas sessões e aos eventos correspondentes.

- `COUNT(DISTINCT e.Cidade)` conta quantas **cidades diferentes** cada orador participou, filtrando no `HAVING` apenas os que atuaram em **mais de uma cidade**.
- A subconsulta com `STRING_AGG` agrega todas as cidades distintas em que o orador participou, separadas por vírgula, mostrando onde ele esteve.
- `GROUP BY` garante que cada orador apareça **uma vez** com seus dados e totais agregados.
- `ORDER BY NumeroCidades DESC, o.Nome ASC` lista primeiro os oradores com mais cidades, ordenando alfabeticamente em caso de empate.

Reflexão Crítica

Esta secção reflete sobre os principais desafios encontrados no design e implementação em SQL Server 2025 Express, com foco nas especificidades do T-SQL e nas boas práticas ensinadas na sebenta.

Desafio 1: Nomenclatura e Naming de Constraints

- **Problema:** Constraints sem nome explícito recebem nomes gerados automaticamente pelo sistema (ex.: DF__Eventos__PrecoInscricao__5AEE82B9), dificultando futuras operações de ALTER TABLE DROP CONSTRAINT.
- **Solução:** Adotámos uma convenção sistemática de nomeação para todas as constraints:

➢ PK_ para chaves primárias

➢ UQ_ para constraints de unicidade

➢ CK_ para checks

➢ FK_ para chaves estrangeiras

➢ DF_ para defaults.

Essa prática facilita manutenção, documentação e legibilidade da base de dados.

Desafio 2: IDENTITY e Inserção Controlada de Dados

- **Problema:** Com IDENTITY(1,1), o SQL Server não permite inserir valores explícitos de ID por padrão, causando erros ao popular a base de dados com IDs específicos.
- **Solução:** Usámos a forma natural de inserção, deixando o IDENTITY gerir automaticamente os valores.

➢ Quando necessário forçar um ID específico (por exemplo, em migrações), utilizámos:

➢ SET IDENTITY_INSERT NomeTabela ON;
INSERT ...
SET IDENTITY_INSERT NomeTabela OFF;

➢ Este procedimento está documentado na sebenta (Cap. 1.6.1) e garante **integridade e controle sobre os IDs**.

Desafio 3: Agregação de Valores com STRING_AGG

- **Problema:** Para agregar nomes numa única célula, é necessário usar uma função específica do SQL Server.
- **Solução:** Utilizámos:
- STRING_AGG(valor, separador) WITHIN GROUP (ORDER BY ...)
- WITHIN GROUP permite **ordenar os valores dentro da agregação**, facilitando a leitura e organização das informações.

Desafio 4: Tipos de Dados VARCHAR vs NVARCHAR

- **Problema:** A distinção entre VARCHAR (ASCII/ANSI) e NVARCHAR (Unicode) é crítica para suportar caracteres especiais (ã, ç, é, etc.).
- **Solução:**
- Optámos por NVARCHAR em todos os campos de texto.
- Para literais Unicode, utilizámos o prefixo N (ex.: N'Congresso').
- Esta prática garante **compatibilidade com caracteres Unicode** e evita problemas de codificação.

Desafio 5: Funções de Data e Formatação

- **Problema:** Funções de data e tempo possuem sintaxe específica no SQL Server, exigindo atenção ao reescrever expressões.
- **Solução:** Mapeámos sistematicamente as funções:

Função	SQL Server 2025
Data atual	`CAST(GETDATE() AS DATE)`
Diferença em minutos	`DATEDIFF(MINUTE, a, b)`
Formatação de data	`CONVERT(VARCHAR, d, 103)`

- Sempre que necessário, utilizámos SYSDATETIME() em vez de GETDATE() para **DATETIME2**, obtendo maior precisão (100 nanosegundos).

CONCLUSÕES

Este projeto demonstrou com sucesso a aplicação prática de conceitos fundamentais de Sistemas de Gestão de Bases de Dados. A base de dados foi projetada seguindo uma metodologia rigorosa, normalizada até à Terceira Forma Normal, implementada com restrições de integridade apropriadas, e validada com dados realistas. As cinco consultas SQL desenvolvidas demonstram a capacidade do sistema de responder a questões de negócio relevantes de forma eficiente.